# A2B Neural Rendering and Desktop Implementation: A Comprehensive Technical Report

February 16, 2026

## Part 1: Comprehensive Overview of A2B and Neural Binaural Decoding

### 1.1 The Evolution of Spatial Audio Reproduction

The quest for realistic spatial audio reproduction—specifically the ability to recreate a three-dimensional sound field over stereo headphones—has historically been a battle between mathematical approximation and perceptual reality. The fundamental challenge lies in the translation of a sound field, captured or synthesized in a spherical format, into the binaural signals that the human auditory system expects. This process, known as binaural decoding, transforms the directionality and diffuseness of sound into interaural time differences (ITD), interaural level differences (ILD), and spectral cues governed by the Head-Related Transfer Function (HRTF).[1]

Traditional approaches to this problem, particularly within the domain of Ambisonics, have relied heavily on Digital Signal Processing (DSP) heuristics. Methods such as Magnitude Least Squares (MagLS) or virtual loudspeaker projection attempt to map Spherical Harmonic (SH) coefficients to HRTF datasets analytically. While mathematically robust in theory, these methods suffer from severe limitations in practice. They often assume plane-wave propagation, neglecting the complex scattering and near-field effects that occur when a sound source is close to the listener's head. Furthermore, to mitigate artifacts arising from finite-order representations (spatial aliasing and truncation errors), these traditional pipelines require complex, manually tuned equalization filters and covariance constraints.[1]

The introduction of **A2B (Ambisonics-to-Binaural)** marks a decisive shift from these heuristic-based methods to a data-driven, end-to-end neural rendering paradigm. Developed by researchers at Meta (Facebook Reality Labs), A2B leverages deep learning to perform a direct, non-linear mapping from Ambisonic signals to binaural audio. By training on massive datasets of paired recordings, the model implicitly learns the complex acoustic interactions of the head and torso, effectively bypassing the limitations of linear decoding matrices.[1]

## 1.2 Theoretical Foundations: The Ambisonics Domain

To understand the innovation of A2B, one must first dissect the input domain: Ambisonics. Unlike channel-based audio (e.g., 5.1 or 7.1), Ambisonics represents the sound field as a series of orthogonal basis functions on a sphere, known as Spherical Harmonics $Y_n^m(\theta, \phi)$. This representation is order-dependent, where the spatial resolution increases with the Ambisonic order $N$.

The number of channels required to represent an Ambisonic field of order $N$ is given by $(N+1)^2$.

- **First Order (FOA):** 4 channels ($W, Y, Z, X$). Represents basic directionality but suffers from low spatial resolution.
- **Higher Order Ambisonics (HOA):** Orders 2 and above. 4th order requires 25 channels; 10th order requires 121 channels. These provide sharper localization but increase data bandwidth significantly.[5]

Standard Ambisonic formats follow specific normalization and channel ordering conventions to ensure interoperability. The A2B research utilizes the **AmbiX** format, which employs **ACN (Ambisonic Channel Number)** ordering and **SN3D (Schmidt Semi-Normalization)**.[5]

| Order (N) | Channel Count | Spatial Resolution | Typical Application |
|---|---|---|---|
| 1st Order | 4 | Low (Cardioid-like) | Zoom H3-VR, basic 360 video |
| 2nd Order | 9 | Medium | VR Gaming, YouTube VR |
| 4th Order | 25 | High | Professional VR, A2B Baseline [1] |
| 10th Order | 121 | Ultra-High | Research, A2B High-Res Models [1] |

The "Ambisonics Gap" refers to the degradation of quality when rendering lower orders. Linear decoders struggle to produce sharp binaural images from 1st or 2nd order inputs because the "energy spread" is too wide. A2B addresses this by using the neural network to

"super-resolve" the spatial cues, hallucinating the missing high-frequency spatial information based on learned priors from high-quality training data.[3]

## 1.3 The A2B Architecture and Methodology

The A2B framework replaces the explicit stages of a traditional decoder (Virtual Speaker selection -> HRTF convolution -> Equalization) with a single neural inference pass.

### 1.3.1 Neural Signal Processing

While the exact layer-by-layer topology is proprietary, the architecture functions as a conditioned audio-to-audio regression model. The input is a multi-channel time-series tensor of shape $(B, C, T)$, where $B$ is batch size, $C$ is the number of Ambisonic channels (e.g., 25 for 4th order), and $T$ is the number of samples. The output is a stereo tensor $(B, 2, T)$.

The network likely employs a **U-Net** or **WaveNet-style TCN (Temporal Convolutional Network)** backbone, designed to preserve phase integrity. Unlike spectrogram-based models that often discard phase information (leading to metallic artifacts), A2B operates in a domain that preserves the temporal fine structure essential for ITD cues.[1]

The model is trained to minimize a composite loss function that includes:

1. **L1/L2 Time-Domain Error:** Ensures waveform alignment.
2. **Multi-Resolution STFT Loss:** Ensures spectral accuracy across different frequency bands.
3. **Binaural Losses:** Specific penalties for errors in Interaural Level Difference (ILD) and Interaural Time Difference (ITD), ensuring the predicted audio localizes correctly in perceptual space.[1]

### 1.3.2 Training on Paired Real-World Data

The efficacy of A2B stems from its training data. The researchers collected a massive dataset of **paired Ambisonic-to-Binaural recordings**. This involved placing a high-order spherical microphone array (e.g., Zylia or Eigenmike) and a binaural mannequin (e.g., Neumann KU-100) at the exact same point in a sound field, then recording various source stimuli (speech, noise, music) from varying distances and angles.[1]

The dataset includes distinct subsets to test different resolutions:

- **10-A2B-R1 / R2:** 10th-order recordings (121 channels) serving as the high-fidelity ground truth proxy.
- **4-A2B-MP:** 4th-order recordings (25 channels).
- **2-A2B-MP:** 2nd-order recordings (9 channels).

This pairing allows the network to learn the "transfer function" of the specific microphone

array and the specific binaural head simultaneously. It learns to compensate for the microphone's self-noise, the array's aliasing filters, and the complex near-field diffraction patterns of the head.[1]

## 1.4 Comparative Performance Analysis

The A2B model demonstrates significant improvements over traditional DSP methods, particularly in challenging acoustic scenarios.

| Metric | MagLS (Traditional DSP) | A2B (Neural Rendering) | Impact / Implication |
|---|---|---|---|
| **Near-Field Accuracy** | Poor. Assumes sources are at infinity (plane waves). Incorrect ILD for close sources. | High. Trained on real near-field data, capturing correct proximity effects (bass boost, high ILD). | Critical for "whisper" effects in VR and intimate social presence.[1] |
| **Order Sensitivity** | High. Quality degrades rapidly below 3rd order. | Low. Can produce plausible binaural cues even from 1st/2nd order inputs by learning spatial priors. | Enables higher quality on bandwidth-constrained platforms (mobile VR).[3] |
| **Phase Coherence** | Variable. Filtering often introduces group delay distortion. | High. End-to-end time-domain optimization preserves transient response. | Improved clarity and transient definition in percussion and speech. |
| **Artifacts** | "Phasiness" and spectral coloring due to order truncation. | Generative artifacts (rare) or slight noise, but generally spectrally balanced. | Smoother listening experience without manual EQ tuning.[1] |

The shift to A2B represents a broader trend in audio engineering: the move from **analytic modeling** (describing physics via equations) to **empirical modeling** (learning physics from data). This is particularly advantageous for reproducing "imperfect" real-world fields, where

analytic models often fail to account for room reflections, microphone body scattering, and non-rigid head movements.[3]

---

# Part 2: Desktop Implementation Plan

## 2.1 Strategic Overview

The objective is to implement a desktop application for macOS that leverages the A2B neural rendering engine to play back Ambisonic audio files. The application must support **arbitrary order input** (up to 10th order), **real-time head tracking** (Yaw/Pitch/Roll) via OSC, and a modern **drag-and-drop** interface.

To achieve low-latency performance suitable for head-tracked binaural audio (motion-to-photon latency < 20ms), the system must utilize hardware acceleration. On macOS, this mandates the use of **Metal Performance Shaders (MPS)** via PyTorch, which allows neural inference and matrix operations to run directly on the Apple Silicon GPU, sharing memory with the CPU via the Unified Memory Architecture.[9]

## 2.2 System Architecture

The application will be architected around a **multi-threaded, ring-buffered pipeline** to decouple the user interface, network listeners, and the audio/compute engine.

### 2.2.1 Component Diagram

1. **Main Thread (GUI - PySide6):** Manages the view layer, processes drag-and-drop events, and visualizes the head orientation.
2. **OSC Receiver Thread (Daemon):** Listens for UDP packets containing head tracking coordinates. Updates a thread-safe atomic state.
3. **Audio/Compute Thread (High Priority):**
    - **Input Stage:** Reads audio frames from disk (SoundFile).
    - **Adaptation Stage:** Zero-pads or truncates input channels to match the model's expected order.
    - **Rotation Stage:** Applies Wigner-D rotation matrices to the Ambisonic coefficients (GPU-accelerated).
    - **Inference Stage:** Runs the A2B neural network (PyTorch MPS).
    - **Output Stage:** Pushes stereo samples to the CoreAudio hardware buffer (SoundDevice).

### 2.2.2 Technology Stack

- **Language:** Python 3.10+ (Required for stable torch.backends.mps).[11]
- **GUI Framework: PySide6**. Chosen for its native macOS look, robust event loop, and compatibility with threaded applications.[12]

- **Compute Backend: PyTorch (MPS)**. Essential for real-time inference on Mac. CUDA is not available, and CPU inference is too slow for high-order Ambisonics at low latency.[10]
- **Audio I/O: SoundDevice (PortAudio wrapper)**. Provides cross-platform, low-latency access to the audio callback.[14]
- **Networking: python-osc**. A zero-dependency library for handling Open Sound Control messages.[15]

## 2.3 The Mathematical Engine: Real-Time Ambisonic Rotation

A critical requirement is head tracking. The A2B model itself is a static decoder—it renders the sound field as captured. To simulate head movement, we must rotate the *entire sound field* relative to the listener before it enters the neural network. This operation is valid because Spherical Harmonics are rotation-invariant.

### 2.3.1 The Wigner-D Matrix approach

Rotation in the Spherical Harmonic domain is achieved by multiplying the vector of SH coefficients $\mathbf{s}$ by a block-diagonal rotation matrix $\mathbf{R}$.

$$\mathbf{s}_{rotated} = \mathbf{R}(\alpha, \beta, \gamma) \cdot \mathbf{s}_{input}$$

where $\alpha, \beta, \gamma$ are the Euler angles (Yaw, Pitch, Roll).

The matrix $\mathbf{R}$ is composed of smaller sub-matrices $\mathbf{D}^{(l)}$ for each Ambisonic order $l$. For a 10th order signal, the total matrix size is $121 \times 121$.

- Order 0 (1 channel): Scalar (invariant).
- Order 1 (3 channels): $3 \times 3$ Rotation Matrix.
- Order $l$ ($2l + 1$ channels): $(2l + 1) \times (2l + 1)$ Wigner-D Matrix.[16]

### 2.3.2 GPU-Accelerated Rotation

Calculating high-order Wigner-D matrices on the CPU and transferring them to the GPU for every audio frame introduces unacceptable latency.

**Implementation Strategy:** We will implement the recursive Wigner-D generation algorithm directly in PyTorch as a nn.Module.

1. **Input:** Quaternion or Euler angles from the OSC thread.
2. **Compute:** Generate the $\mathbf{D}^{(l)}$ matrices on the MPS device.
3. **Apply:** Perform batch matrix multiplication (torch.bmm) with the audio tensor on the

MPS device. This keeps the entire signal path (Audio -> Rotate -> Inference) within the GPU memory, minimizing PCIe/Unified Memory bus traffic.[18]

## 2.4 Handling Arbitrary Orders

The A2B model is trained for a specific input configuration (e.g., 4th order, 25 channels). User files may vary. The application must implement an **Adaptation Layer**:

- **Case: File Order < Model Order:**
  - *Action:* **Zero-Padding**. If the model expects 25 channels but the file has 4 (1st order), we fill channels 5-25 with zeros.
  - *Implication:* The model will render a lower-resolution spatial image, but the neural weights for higher orders will simply not activate. This effectively mimics up-sampling.[6]
- **Case: File Order > Model Order:**
  - *Action:* **Truncation**. If the file is 10th order but the model is 4th, we discard channels 26-121.
  - *Implication:* Loss of spatial detail, but valid playback.
- **Case: File Order = Model Order:** Direct pass-through.

## 2.5 macOS Implementation Specifics

### 2.5.1 The Drag-and-Drop Workaround

PySide6 on macOS has a known issue where dragging a file from Finder often results in a generic file:// reference or internal ID that Python's open() cannot read.

- **Solution:** We must use the pyobjc library to interface with the native Cocoa NSURL class.
- **Mechanism:** When the dropEvent occurs, we extract the raw URL bytes, pass them to NSURL, and call .filePathURL().path() to resolve the absolute POSIX path (e.g., /Users/name/Music/test.amb).[20]

### 2.5.2 Audio Permissions and Threading

macOS requires strict permissions for microphone access (even for output-only streams, sounddevice often triggers the entitlement check).

- **Entitlements:** If packaging as an .app, the Info.plist must include NSMicrophoneUsageDescription.
- **GIL Management:** Python's Global Interpreter Lock (GIL) can cause audio dropouts if the GUI thread blocks the audio thread. We will use QThread with QThread.TimeCriticalPriority for the audio engine, and ensure all heavy math is released to the C++ backend of PyTorch.[21]

### 2.5.3 OSC Coordinate Mapping

Head trackers and Ambisonics use different coordinate systems.

- **Ambisonics (ACN/N3D):** Cartesian, Right-Handed. +X = Front, +Y = Left, +Z = Up.
- **Head Trackers:** often Aviation (Yaw/Pitch/Roll).
- **Mapping:** The OSC thread must normalize the input range (typically -180 to 180 or 0 to 360) to Radians and construct the appropriate rotation quaternion.
  - Yaw (Head Shake) -> Rotation around Z-axis.
  - Pitch (Nod) -> Rotation around Y-axis.
  - Roll (Tilt) -> Rotation around X-axis.
  - The application must allow users to "zero" the tracker (set current orientation as Forward).[16]

---

# Part 3: Product Requirements Prompt (PRP) for AI Agent

**Agent Persona:** You are an autonomous Senior Software Engineer specializing in Python, Qt, and Audio DSP.

**Environment:** Antigravity IDE (VS Code fork).

**Methodology:** Strict Test-Driven Development (TDD). You do not write implementation code until a test has failed.

**Goal:** Build "NeuroAmbi," a macOS desktop application for neural Ambisonic rendering.

## Phase 1: Environment & Core Audio Architecture

**Goal:** Establish a stable PyTorch MPS environment and a basic audio playback engine.

1. **Project Initialization & Dependency Validation**
   - **Prompt:** "Initialize a new Python project. Create a virtual environment. Create a requirements.txt containing: pyside6, torch, sounddevice, soundfile, numpy, python-osc, pyobjc-framework-Cocoa.
   - **Test 1.0 (Environment):** Create tests/test_env.py. Write a test case test_mps_availability that imports torch and asserts torch.backends.mps.is_available() is True.
     - *Context:* This confirms we are running on Apple Silicon with access to the Neural Engine.
   - **Action:** Run the test. If it fails, halt and request environment troubleshooting.
2. **Audio Loading & Buffering**
   - **Prompt:** "We need to handle audio files efficiently. Implement a file loader that supports.wav."
   - **Test 1.1 (Loader):** Create tests/test_audio_io.py. Write test_load_wav that creates a temporary 1-second sine wave.wav file, loads it using your AudioLoader class, and asserts the output shape is (Samples, Channels) and dtype is float32.

- **Test 1.2 (Circular Buffer):** Write test_ring_buffer. Initialize a ring buffer of size 4096. Write 2048 samples. Read 1024. Assert the read pointer advanced correctly.
  - *Context:* Neural inference is block-based (e.g., 1024 samples), but audio callbacks are hardware-driven (e.g., 512 samples). A ring buffer is mandatory to bridge this gap without under-runs.

3. **Audio Output Engine**
   - **Prompt:** "Create the engine that talks to the hardware."
   - **Test 1.3 (Callback):** Create tests/test_engine.py. Mock sounddevice.OutputStream. Initialize the AudioEngine. Assert that when start() is called, the mock stream is opened. Assert that the callback function consumes data from the ring buffer.

## Phase 2: The Math Engine (GPU Rotation)

**Goal:** Implement real-time Spherical Harmonic rotation on the GPU.

1. **Rotation Matrix Generation**
   - **Prompt:** "We need to rotate the sound field. We will use Euler angles (Yaw, Pitch, Roll)."
   - **Test 2.1 (Matrix Shape):** Create tests/test_rotation.py. Define a RotationLayer(nn.Module). Instantiate it for Order 1 (4 channels). Input dummy angles (0, 0, 0). Assert output matrix is the $4 \times 4$ Identity matrix.
   - **Test 2.2 (Functional Rotation):** Generate a synthetic 1st-order signal representing a source at 0° (Front): $W = 1, X = 1, Y = 0, Z = 0$. Apply a Yaw of 90° (Left). Assert the output is approx $W = 1, X = 0, Y = 1, Z = 0$.
     - *Context:* Use torch.testing.assert_close for floating point comparisons.
2. **Higher Order Recursion**
   - **Prompt:** "Extend the rotation to arbitrary orders using Wigner-D recursion."
   - **Test 2.3 (HOA Support):** Instantiate RotationLayer for Order 3 (16 channels). Pass a batch of random angles. Assert output tensor shape is (Batch, 16, 16). Check gradients are enabled (though we won't train, it confirms it's a valid graph).
   - **Action:** Implement the recursive algorithm on PyTorch tensors. *Constraint:* Do not use pyshtools or CPU libraries; this must run on device='mps'.

## Phase 3: Neural Integration (A2B Model)

**Goal:** Integrate the A2B inference pass into the audio loop.

1. **Model Loading & Adaptation**
   - **Prompt:** "We need to load the A2B model checkpoint."
   - **Test 3.1 (Model Input):** Create tests/test_model_integration.py. specificy a mock model class MockA2B that mimics the input/output signature of the real model. Write a test that initializes the model with order=4 (25 channels). Pass an input tensor of shape (1, 25, 1024). Assert output shape (1, 2, 1024).

- **Test 3.2 (Adaptation Logic):** Test the adapt_input(tensor, target_order) function.
  - Case A: Input 4 channels, Target 25. Assert output is 25 channels with zero-padding.
  - Case B: Input 49 channels, Target 25. Assert output is 25 channels (truncated).

2. **Real-Time Inference Loop**
   - **Prompt:** "Connect the AudioEngine to the Model."
   - **Test 3.3 (Latency Check):** Benchmark test. Load the real model (or a heavy dummy). Measure the execution time of model(tensor) on MPS. Assert execution time < 10ms for a 20ms buffer.
     - *Context:* If this fails, we cannot run real-time. We may need to use torch.jit.trace to optimize the model graph.

## Phase 4: Control Systems (OSC & Head Tracking)

**Goal:** Enable remote control of the rotation angles.

1. **OSC Server Implementation**
   - **Prompt:** "Implement the OSC listener."
   - **Test 4.1 (Message Parsing):** Create tests/test_osc.py. Create an OSCController class. Simulate an incoming UDP packet for address /yaw with argument 45.0. Assert controller.yaw state updates to 45.0.
   - **Test 4.2 (Coordinate Mapping):** Input Yaw=90, Pitch=0, Roll=0. Assert the controller outputs the correct Euler angles in Radians for the RotationLayer (e.g., [1.57, 0, 0]).
2. **Thread Safety**
   - **Test 4.3 (Concurrency):** Simulate the OSC thread writing angles while the Audio thread reads them. Assert no race conditions or partial updates (use atomic primitives or a lightweight threading.Lock).

## Phase 5: GUI & User Experience

**Goal:** Build the visual interface.

1. **Main Window & Drag-and-Drop**
   - **Prompt:** "Create the PySide6 Main Window."
   - **Test 5.1 (UI Launch):** Create tests/test_gui.py. Initialize QApplication. assert MainWindow exists.
   - **Test 5.2 (Drag & Drop Logic):** Simulate a QDragEnterEvent with a file URL. Assert event.isAccepted() is True.
   - **Test 5.3 (MacOS URL Fix):** Mock the MimeData object to return a macOS-style file reference. Implement the pyobjc / NSURL resolution logic. Assert the returned path is a valid POSIX string.
2. **Visualization**
   - **Prompt:** "Add a visual indicator for head direction."
   - **Action:** Add a QFrame or QOpenGLWidget that draws a simple compass. Connect

the OSCController signals to the widget's update() slot.

## Phase 6: Packaging & Deployment

**Goal:** Create a distributable.app bundle.

1. **Build Configuration**
   - **Prompt:** "Create a setup.py for py2app."
   - **Action:** Configure py2app options. Critical: Include libtorch and sounddevice dynamic libraries. Add NSMicrophoneUsageDescription to the plist dict in setup options (required for CoreAudio initialization).
   - **Test 6.1 (Build):** Run python setup.py py2app. Verify the dist/NeuroAmbi.app bundle exists.

---

## Conclusion

This report provides a roadmap for bridging the gap between cutting-edge neural audio research and practical desktop application development. By combining the data-driven fidelity of the A2B model with the hardware acceleration of Apple Silicon and the flexibility of the Ambisonics format, the proposed system offers a robust platform for next-generation immersive audio evaluation. The TDD-based Product Requirements Prompt ensures that the autonomous agent constructs the application with architectural rigor, ensuring stability and performance in a complex, multi-threaded, real-time environment.

### Works cited


1. A2B: Neural Rendering of Ambisonic Recordings to Binaural - Israel D. Gebru, accessed February 16, 2026, https://idgebru.com/paper/ICASSP2025_Arxiv___A2B.pdf
2. (PDF) Introduction to Ambisonics - ResearchGate, accessed February 16, 2026, https://www.researchgate.net/publication/280010078_Introduction_to_Ambisonics
3. A2B: Neural Rendering of Ambisonic Recordings to Binaural | Request PDF - ResearchGate, accessed February 16, 2026, https://www.researchgate.net/publication/390537390_A2B_Neural_Rendering_of_Ambisonic_Recordings_to_Binaural
4. facebookresearch/A2B: A2B Neural Rendering of ... - GitHub, accessed February 16, 2026, https://github.com/facebookresearch/A2B
5. Ambisonics for Beginners - Masterklasse Audio Design, accessed February 16, 2026, https://audiodesign.fhstp.ac.at/wp-content/uploads/2020/07/AmbisonicsTutorial_Beginners_200701.pdf
6. python-acoustics/acoustics/ambisonics.py at master - GitHub, accessed February 16, 2026, https://github.com/python-acoustics/python-acoustics/blob/master/acoustics/am

bisonics.py
7. HARP: A Large-Scale Higher-Order Ambisonic Room Impulse Response Dataset - arXiv, accessed February 16, 2026, https://arxiv.org/pdf/2411.14207
8. Alexander Richard - Research Scientist - Meta Reality Labs Research, accessed February 16, 2026, https://alexanderrichard.github.io/
9. Accelerated PyTorch training on Mac - Metal - Apple Developer, accessed February 16, 2026, https://developer.apple.com/metal/pytorch/
10. Introducing Accelerated PyTorch Training on Mac, accessed February 16, 2026, https://pytorch.org/blog/introducing-accelerated-pytorch-training-on-mac/
11. Setting up PyTorch on Mac M1 GPUs (Apple Metal / MPS) - geekmonkey, accessed February 16, 2026, https://geekmonkey.org/setting-up-jupyter-lab-with-pytorch-on-a-mac-with-gpu/
12. Drag and Drop widgets in PySide6 with this Drop-in Sortable Widget - Python GUIs, accessed February 16, 2026, https://www.pythonguis.com/faq/pyside6-drag-drop-widgets/
13. PySide6 Tutorial: Building GUI Applications with Python - DataCamp, accessed February 16, 2026, https://www.datacamp.com/tutorial/introduction-to-pyside6-for-building-gui-applications-with-python
14. Threading · Issue #118 · spatialaudio/python-sounddevice - GitHub, accessed February 16, 2026, https://github.com/spatialaudio/python-sounddevice/issues/118
15. Python-osc - OSC server and client in pure python — python-osc 1.7.1 documentation, accessed February 16, 2026, https://python-osc.readthedocs.io/
16. JSAmbisonics: A Web Audio library for interactive spatial sound processing on the web 1 Introduction - University of York, accessed February 16, 2026, https://www.york.ac.uk/sadie-project/IASS2016/IASS_Papers/IASS_2016_paper_16.pdf
17. Algorithms for Computing Ambisonics Translation Filters 1 Definitions and conventions - The 3D3A Lab at Princeton University, accessed February 16, 2026, https://3d3a.princeton.edu/document/76
18. Ambisonic Spatial Transformations - MATLAB & Simulink - MathWorks, accessed February 16, 2026, https://www.mathworks.com/help/audio/ug/ambisonic-spatial-transformations.html
19. spaudiopy.sph — spaudiopy v0.2.0-1-g8995ca8 documentation - spaudiopy's documentation!, accessed February 16, 2026, https://spaudiopy.readthedocs.io/en/latest/spaudiopy.sph.html
20. Pyside example of dragging and dropping a file onto a GUI. See http://www.birving.com/blog/2015/09/29/drag-and-drop-files-into-gui-using/ · GitHub, accessed February 16, 2026, https://gist.github.com/benjaminirving/f45de3bbabbcacd3ca29
21. PySide6 GUI with Thread-Safe Plotting : r/QtFramework - Reddit, accessed February 16, 2026,

https://www.reddit.com/r/QtFramework/comments/1cywxjn/pyside6_gui_with_threadsafe_plotting/
22. Rotations - Blue Ripple Sound, accessed February 16, 2026, http://www.blueripplesound.com/notes/rotations
23. ADM-OSC - GitHub Pages, accessed February 16, 2026, https://immersive-audio-live.github.io/ADM-OSC/